The Exploitability Gap in LLM-Serving Scheduler Portfolios

Soroush Vahidi[1]

New Jersey Institute of Technology, Newark, NJ, USA
sv96@njit.edu

Abstract. LLM-serving schedulers have workload-dependent strengths, making portfolios attractive. We study a six-policy portfolio across controlled mechanisms, a 240-scenario joint workload, pre-specified adaptive-exploitation tests, and local vLLM validation. Complementarity persists: the per-scenario best policy retains 0.0190 arrival-normalized weighted-goodput improvement over the best fixed policy. Yet this headroom is not deployable gain: selectors, routing, support expansion, guarded composition, and typed synthesis do not satisfy pre-specified utility or robustness criteria despite detectable regime structure. Real-vLLM results add a systems lesson: native vLLM semantics do not directly match the simulator abstraction, but expose a workload-dependent token-budget tradeoff.

Keywords: LLM serving · scheduling · portfolios · vLLM

1 Introduction

LLM inference serving has to schedule heterogeneous work. Requests differ in prompt length, predicted and realized generation length, SLO or deadline tightness, class or tenant weight, KV-cache footprint, burst timing, and interactions between prefill and decode. These dimensions stress different mechanisms in the serving stack: one workload may reward uninterrupted prompt processing, another may reward urgent-deadline protection, and another may reward preserving scarce KV capacity. It is therefore unlikely that one simple scheduling rule will dominate all operating regimes.

This makes scheduler portfolios appealing, echoing algorithm-selection work that compares a single best solver against a virtual best solver (SBS versus VBS) [20, 7, 29, 13]. We adopt that perspective for LLM-serving schedulers. The VBS chooses the best policy independently for each scenario, while an online scheduler must act from causal observations under queues whose future state depends on earlier decisions. We use *exploitability gap* for the residual portfolio opportunity left uncaptured by a tested adaptive mechanism.

The gap is nontrivial even when regimes are statistically detectable. A router or selector may identify the broad workload family while still failing to recover utility, because minority high-value intervals are washed out, decision-state support shifts under closed-loop execution, or the features that separate regimes do

not identify the action that should be taken at a specific queue state. Thus, the relevant question is not only whether policies are complementary, but whether that complementarity is available to a practical adaptive scheduler under the serving system's causal information.

We study this question with a fixed six-policy LLM-serving portfolio. The policies cover full and chunked prefill control, estimated-service ordering, weighted fair sharing, least-laxity urgency, and KV-aware admission, drawing motivation from LLM serving systems and mechanisms such as iteration-level scheduling, PagedAttention, chunked prefill, disaggregated prefill/decode serving [30, 12, 2, 31], and token-level fairness mechanisms [23]. We evaluate the portfolio on controlled mechanism families, a broader joint multi-mechanism workload distribution, and local real-vLLM experiments. We then test a sequence of increasingly strong exploitation attempts: contextual selection, shared-feature selection, hierarchical routing, target-free support expansion, guarded mechanism composition, and exact-parent typed structural synthesis.

The evidence separates opportunity from exploitation. In the joint workload, 59.6% of scenarios have an ϵ-unique winner at $\epsilon = 0.01$, and the six-policy VBS improves mean ANWG by 0.019034 over the best fixed policy. But a hierarchical router with macro-F1 0.9887 achieves only +0.00616 live mean ANWG, below the pre-specified +0.01 practical-effect criterion, and portfolio structural crossover finds best mean marginal gain 0.0 under an equal training-only budget. Real-vLLM validation gives a parallel systems warning: a direct simulator-to-vLLM prefill/decode analogue fails because native vLLM scheduling semantics differ, while native vLLM's own token-budget control shows a stable latency tradeoff.

This paper makes four contributions:

1. We show workload-dependent complementarity and positive VBS headroom for a six-policy LLM-serving portfolio, including under jointly varying synthetic workload pressures.
2. We specialize VBS-versus-realized-selector methodology to LLM-serving portfolios and show that strong regime detectability does not yield sufficient closed-loop gain for the tested selector/router formulations.
3. We test support expansion, guarded mechanism composition, and strongly typed scheduler synthesis; none satisfies its pre-specified robustness or transfer criterion under the tested formulation.
4. We validate a simulator-derived mechanism against native vLLM, identify a semantic mismatch, and isolate a repeatable native scheduler-token-budget tradeoff in the tested configuration.

2 Portfolio, Workloads, and Metrics

2.1 Problem setting

We model an LLM-serving instance as an online scheduling problem over arriving requests. Each request has an arrival time, prompt-token length, predicted

generation length, class or tenant identifier, priority weight, and SLO deadline. During execution, the simulator and serving system expose causal state such as queued requests, admitted/running requests, elapsed waiting time, remaining or predicted service proxies, active sequence counts, prefill/decode phase information when modeled, and KV capacity or pressure. A scheduling policy chooses actions such as which request to admit or serve next, how prompt work is chunked when that control exists, and whether KV/load constraints allow admission.

The policy input is intentionally online-observable. The future actual output length of an unfinished request is not exposed; it is used only after a run to compute outcomes. Thus offline VBS quantities, including the best policy for a scenario, are analytical baselines rather than scheduler inputs.

2.2 Six-policy portfolio

The fixed portfolio contains six policies spanning distinct mechanisms. Table 1 summarizes the paper-facing interpretation; implementation details remain in the repository artifacts.

Table 1. Six-policy portfolio used for envelope analysis.

Policy	Mechanism	Key online state
Full prefill	Uninterrupted prompt processing for long prompt-heavy work	Queue order, prompt phase, prefill budget
Small-chunk prefill	Prompt chunking for prefill/decode interleaving and late TTFT	Queue order, prompt phase, chunk budget
ESTF	Short estimated-service first for completion efficiency	Prompt tokens, predicted output tokens
WFS	Weighted service-deficit fairness and SLO protection	Class weights, admitted/completed service
LLF	Least-laxity urgency under deadlines	Deadline, current time, predicted service
KV constrained	KV-reserve admission/scheduling with urgent bypass	KV capacity, estimated KV demand, laxity

These are repository-specific baselines, not full reproductions of external systems. `full_prefill` is motivated by iteration-level batching in Orca and vLLM [30, 12]; `chunked_prefill_small` by chunked-prefill work such as Sarathi-Serve [2], though our Family-B simulator does not implement Sarathi-Serve's complete stall-free decode-priority algorithm. ESTF follows classical size-based SPT/SRPT intuition [22]; WFS is grounded in GPS/WFQ-style fairness [16, 5] and related LLM-serving fairness work [23]; LLF is a real-time urgency heuristic; and the KV policy is related to KV-aware serving [12, 9].

2.3 Workload suites

We use four workload categories. First, public trace replay provides a realism baseline using BurstGPT and Azure 2023 LLM-inference traces [25, 14, 17]. We construct 20 200-request windows from each of BurstGPT, Azure conversation,

and Azure code, retaining source order, relative timestamps, and prompt/output token counts; predicted lengths, deadlines, priorities, and classes are deterministic overlays, and actual future output length is hidden from online policies. This tested replay configuration motivates stress workloads but does not show that public traces are generally uninformative.

Second, controlled Families A/B/C isolate mechanisms. Family A stresses fairness, completion, and SLO conflict; Family B stresses prefill/decode contention; and Family C stresses KV pressure and urgency. These families form the original controlled evidence for policy complementarity and the later selector, router, support-expansion, and synthesis tests.

Third, the joint multi-mechanism workload broadens the evidence beyond concatenated stress families. It contains 240 scenarios drawn from one common schema that jointly varies offered load, burstiness, long-vs-short mixture, prompt-length heterogeneity, output/service-length heterogeneity, tenant weight skew, SLO tightness, prediction noise, KV pressure, late-arrival structure, active-sequence capacity, and step-token budget. Its pressure audit is outcome-independent: it computes six normalized indicators in $[0, 1]$ covering fairness, service heterogeneity, prefill/decode contention, KV pressure, urgency, and burst pressure. A pressure is counted as elevated when the corresponding indicator is at least 0.60, as specified before policy evaluation. Under this audit, 223 of 240 scenarios (92.9%) have at least two elevated mechanism pressures, and 175 have at least three.

Fourth, we run local real-vLLM validation on Qwen2.5-0.5B served by vLLM 0.27.1 on an RTX 5060 Ti. Those experiments are not used to tune the simulator. They test whether the simulator's prefill/decode abstraction corresponds to native vLLM behavior and whether native vLLM exposes its own scheduling-control tradeoff.

Across these studies, practical-effect and robustness criteria were specified before the corresponding held-out or screening evaluations. Selector/router analyses distinguish offline labels from live closed-loop re-evaluation. The joint-workload CI uses 1,000 scenario-bootstrap resamples with replacement; the native-vLLM CIs use percentile bootstraps over five paired repetition-level differences after matched warmups and ABBA treatment ordering.

2.4 Metric and envelope definitions

The primary simulator utility is arrival-normalized weighted goodput (ANWG). For a scenario with arriving requests i, priority weights w_i, completion times C_i, and deadlines D_i, Eq. 1 defines the implemented metric:

$$\mathrm{ANWG} = \frac{\sum_i w_i \mathbf{1}[i \text{ completed}] \mathbf{1}[C_i \le D_i]}{\sum_i w_i}. \tag{1}$$

Requests that are rejected, dropped, unfinished, or completed after their SLO deadline receive zero numerator credit but remain in the arrival-weight denominator. This differs from an older conditional weighted-goodput metric whose denominator considered only completed requests.

Arrival normalization is important because scheduling policies can change both which requests finish and which finished requests meet SLO. Weighting matters because the workload families include class or tenant priorities. The metric is therefore a system-level weighted SLO goodput, not merely a conditional success rate among completed requests.

For an evaluation set X and fixed portfolio P_6, Eqs. 2 –4 define the single best scheduler (SBS), the virtual best scheduler (VBS), and VBS headroom:

$$h^* = \arg\max_{h \in P_6} \frac{1}{|X|} \sum_{x \in X} R_h(x), \tag{2}$$

where $R_h(x)$ is the ANWG achieved by policy h on scenario x.

$$E_6(x) = \max_{h \in P_6} R_h(x). \tag{3}$$

$$\frac{1}{|X|} \sum_{x \in X} E_6(x) - \frac{1}{|X|} \sum_{x \in X} R_{h^*}(x). \tag{4}$$

It measures opportunity in the portfolio, not deployable adaptive performance. We call Eq. 4 VBS headroom, and $E_6(x) - R_{h^*}(x)$ per-scenario VBS gain. A policy is an ϵ-unique winner when $R_h(x) \geq \max_{q \neq h} R_q(x) + \epsilon$; here $\epsilon = 0.01$ ANWG. Selector regret is mean $E_6(x) - R_{\hat{h}(x)}(x)$ for selected policy $\hat{h}(x)$. VBS-gap closure is (adaptive mean - SBS mean)/(VBS mean - SBS mean). A synthesized candidate's marginal gain is $\max(R_c(x), E_6(x)) - E_6(x)$, averaged over the prespecified training screen.

3 Complementarity Beyond Handcrafted Families

The first requirement for an adaptive scheduler portfolio is simple: the parents must disagree in useful ways. We therefore separate two questions. Do the policies produce workload-dependent winners under controlled mechanisms? And does that complementarity persist when mechanisms vary jointly rather than as three disjoint stress families?

3.1 Controlled mechanism families

The controlled A/B/C suites isolate specific serving tensions. Family A varies fairness, completion, and SLO conflict; Family B varies prompt-heavy prefill pressure against late short requests; and Family C varies KV pressure and urgent arrivals. Across the unified 176-scenario, six-policy matrix, the portfolio produces 1,056 policy cells and a 54.0% ϵ-unique-winner rate. The best fixed policy and VBS headroom differ by family: WFS is best fixed in Family A with 0.021742 headroom, full prefill is best fixed in Family B with 0.049433 headroom, and `kv_constrained_online` is best fixed in Family C with 0.020261 headroom. These results are not used as a production traffic claim; their role is mechanism isolation.

3.2 Public-trace saturation

Public-trace replay is a sanity check, not the main discriminating benchmark. Under the processed replay configuration, all 60 windows tie at ANWG 1.0, six-policy envelope gain is 0.0, active count p99 is 5 out of 512, and maximum KV utilization is 0.003802. This means only that this replay configuration does not expose scheduler differentiation for the current objective.

3.3 Joint multi-mechanism generalization

To test whether complementarity is merely an artifact of disjoint handcrafted families, we generated a 240-scenario joint workload from one common schema before evaluating policy outcomes. The run produced 1,440 successful policy-scenario cells. The outcome-independent coverage audit shows that 223 scenarios (92.9%) have at least two elevated mechanism pressures, and 175 have at least three.

Figure 1 summarizes the main result. Per-scenario winners remain distributed across the portfolio: LLF wins 59 scenarios, `kv_constrained_online` 50, full prefill 46, ESTF 45, WFS 35, and small-chunk prefill 5. The ϵ-unique-winner fraction is 59.6%, nontrivial policy spread appears in 91.7% of scenarios, and the mean policy range is 0.111658 ANWG.

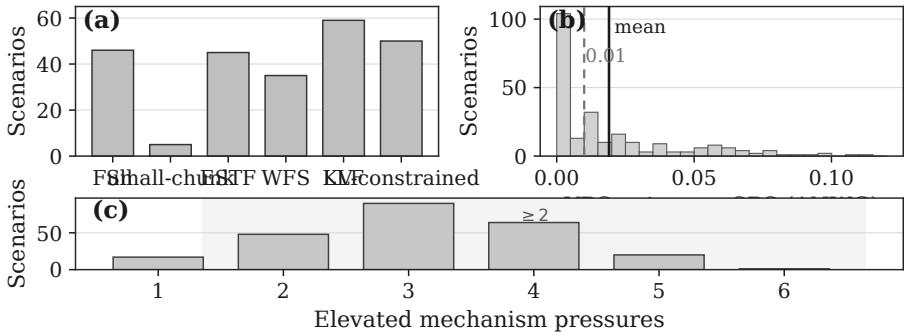

Fig. 1. Joint 240-scenario multi-mechanism workload. (a) Per-scenario VBS winner counts. (b) Per-scenario VBS gain over the SBS. (c) Elevated mechanism pressures per scenario (Section 2). Winner diversity and positive VBS headroom persist under jointly varying pressures.

The SBS over the joint workload is `kv_constrained_online`, with mean ANWG 0.314072. The six-policy VBS reaches mean ANWG 0.333106, yielding VBS headroom 0.019034 with bootstrap 95% CI [0.015988, 0.022433]. The median per-scenario VBS gain is 0.010622, p90 gain is 0.058040, 60.4% of scenarios have positive gain, and 51.3% have gain at least 0.01. The gain is not only located at old single-mechanism extremes: 93.1% of total VBS gain comes from

scenarios with at least two elevated mechanism pressures, and 63.3% comes from scenarios with at least three. The top 10% of scenarios account for 40.5% of the gain, so the opportunity is concentrated but not reduced to a single outlier.

Thus, within this broader synthetic distribution, the portfolio retains meaningful complementarity and positive VBS headroom. The positive complementarity result is therefore not solely an artifact of concatenating three isolated stress families. It remains bounded evidence, not a claim about arbitrary production workloads.

3.4 From opportunity to exploitability

VBS headroom is an opportunity measure. It does not specify which causal features identify the winning action online, whether those features remain stable under closed-loop execution, or whether a practical selector can recover the sparse decisions that produce most of the utility difference. The rest of the paper therefore asks how much of this opportunity the tested adaptive methods can actually realize.

4 The Exploitability Gap

The positive results above make a naive adaptive story tempting: if policies win in different regimes, train a selector or router to choose the right one. This section shows why that story is incomplete under the evaluated workloads. The tested methods often find statistical structure, but they do not satisfy the pre-specified utility criteria.

4.1 Contextual selection

The first test used the unified 176-scenario matrix to train contextual selectors over the six policies. Within-family holdouts showed real learnability: Family A and B could be selected with zero regret in that setting, and Family C remained competitive. Pooled cross-family selection was the failure point. The best pooled model's regret was 0.0463, worse than the best-fixed baseline at 0.0233 and the trivial majority baseline at 0.0127. Leave-one-family-out transfer was also unstable: the Family-A held-out case had regret 0.4786 versus 0.0767 for the fixed baseline.

A shared-feature audit then asked whether the failure was only an artifact of family-prefixed features. It found a 17-feature zero-missingness schema and confirmed that family identity remained perfectly classifiable, while cross-family nearest neighbors were not more utility-consistent than random. This does not prove that no feature representation can work. It does show that the tested shared-feature rescue did not convert VBS complementarity into a robust cross-family selector.

4.2 Regime detectability versus utility

The hierarchical router was a stronger version of the selection idea. Its Stage-1 regime detector achieved macro-F1 0.9887 over the present classes, with no catastrophic misrouting in the pre-specified rescore. Nevertheless, the live closed-loop re-evaluation gained only +0.00616 mean ANWG over the best global fixed policy, below the pre-specified +0.01 practical-effect criterion. Its VBS-gap closure, analogous to closed-gap reporting in algorithm selection, was 0.143, far below the 0.75 target.

This is the clearest expression of the exploitability gap: a method can detect regimes accurately while still failing to harvest enough scheduling utility. The mismatch is not surprising in hindsight. Regime labels are coarser than local action-value boundaries, and many high-value decisions occur in minority intervals within a scenario. Closed-loop scheduling also changes the future queue and resource state, so a correct-looking route at the scenario level need not preserve the state distribution under which the specialized policy was valuable.

4.3 Why classification and utility differ

The repository evidence supports several non-exclusive explanations. First, high classifier accuracy can be dominated by easy or frequent states whose policy choice has little marginal utility. Second, the utility advantage of a specialized policy can be sparse: missing a small number of critical intervals may erase most of the VBS headroom. Third, closed-loop action effects matter because scheduling choices alter waiting times, active sequences, KV pressure, and subsequent arrivals' effective urgency. Finally, support diagnostics show that expanding observed disagreement states in training does not guarantee movement toward held-out decision states.

We avoid stronger claims. The evidence does not show an impossibility theorem for adaptive scheduling. It shows that, under these evaluated workloads and pre-specified criteria, detectable structure and VBS complementarity were insufficient for the tested adaptive mechanisms.

4.4 Pre-specified-criterion summary

Table 2 condenses the main evaluation chain. The point is not that every method failed for the same reason. Rather, the exploitability gap survived increasingly direct attempts to close it.

The table reports pre-specified criteria rather than post-hoc adjusted metrics. It also distinguishes bounded negative evidence from negative universality claims. The tested methods did not satisfy the criteria, but this does not prove that all adaptive or synthesized schedulers must fail.

Table 2. Summary of pre-specified evaluation criteria for tested adaptive exploitation methods.

Method	Positive intermediate signal	Evaluation result
Pooled/shared selector	Within-family learnability; shared zero-missingness schema	Pooled regret 0.0463 versus best-fixed 0.0233 and majority 0.0127
Hierarchical router	Macro-F1 0.9887; catastrophic misrouting 0	+0.00616 live mean ANWG $<$ +0.01 criterion; VBS-gap closure 0.143
Target-free support expansion	24,314 behavioral fingerprints; 156 new training-side support domains	1/104 held-out development states closer; p90 improvement 0; none of eight pre-specified support criteria satisfied
Guarded ESTF/WFS composition	Best rule reduced WFS regret by 3.11%	Below 10% minimum-effect criterion and violated mechanism-quadrant ordering
Typed structural crossover	All six parents reproduced exactly; 4,320 training candidate-scenario evals	No structural-crossover candidate improved the portfolio envelope; best mean marginal gain 0.0

5 Constructive Falsification Beyond Selection

We next ask whether the gap can be closed by moving beyond direct parent selection. Each experiment below strengthens the exploitation mechanism while using pre-specified criteria and avoiding post-hoc threshold changes.

5.1 Support expansion

The support-expansion hypothesis was that selector failure came from insufficient training-side coverage near important held-out Family-A decision states. A target-free support-expansion sweep produced 24,314 behavioral fingerprints and 156 new training-side support domains without using held-out development states as acquisition targets. The held-out support audit barely moved: under the primary feature-space nearest-neighbor support metric, only 1 of 104 development states moved closer to training support and p90 improvement was 0%. This rejects insufficient training-side support, in the tested expansion formulation, as a sufficient explanation for the gap. It does not show that additional data or dataset-aggregation-style interventions can never help.

5.2 Guarded semantic composition

A second explanation is that choosing one parent policy is too coarse. Family A showed ESTF/WFS complementarity: WFS is a robust fairness and SLO default, while ESTF can protect completion-sensitive work. We therefore tested predeclared WFS-default composite rules with explicit ESTF release guards, avoiding unconstrained scalar blending. The best rule reduced WFS regret by 3.11%, but this was below the pre-specified 10% minimum-effect criterion and it violated the required completion-versus-SLO quadrant ordering. The result is a bounded rejection of this WFS-safe ESTF-release formulation, not a claim that all semantic policy composition fails.

5.3 Typed program synthesis

The strongest exploitation attempt synthesized scheduler programs. The system used a small strongly typed grammar, canonical genome strings, deterministic hashes, type-compatible crossover, bounded mutation, and behavioral fingerprints. Before screening, all six parent genomes had to reproduce their original policies exactly. This instantiates GP and strongly typed GP ideas [11, 15, 26] in an LLM-serving domain, but does not claim novelty for GP, hyper-heuristics, or program evolution [4, 21, 10].

The pre-specified screen compared random grammar search, parent-seeded mutation-only search, and portfolio-seeded structural crossover on the same 24 training scenarios. Each treatment received 60 evaluated candidates, for 4,320 candidate-scenario evaluations total. Random grammar search found a best mean marginal gain of 0.011295 over the six-policy envelope; mutation-only reached 0.002551; structural crossover reached 0.0. The random candidate was not eligible for promotion: despite six ϵ-unique wins on training scenarios, it did not satisfy the pre-specified robustness criteria because gains were concentrated, worst-group regression exceeded the allowed bound, and active behavior was largely WFS-like after ablation. Thus portfolio-aware structural crossover did not expand the envelope in this equal-budget screen, and no synthesized child satisfied the promotion criteria.

6 Real-Serving Semantic Validation

Simulator evidence is useful only when modeled mechanisms correspond to serving-engine semantics. We therefore validated prefill/decode contention on local vLLM. The goal was qualitative mechanism transfer, not exact simulator magnitude matching.

6.1 Native feasibility

The real-system experiments used vLLM 0.27.1 serving Qwen2.5-0.5B on a single NVIDIA RTX 5060 Ti with 16 GB-class VRAM. This was sufficient for controlled prefill/decode and scheduler-semantics probes with streaming TTFT instrumentation, but not for production-scale or high-KV-pressure claims.

6.2 Direct Family-B analogue

Family B holds a shared 512-token simulator step budget fixed and changes only prefill chunk size: full prefill uses maximum chunk 65,536 and small-chunk prefill uses chunk 64, both with decode-first disabled. The native analogue compared chunking disabled with a 4096-token budget against chunking enabled with a 512-token budget. It completed 40/40 regime-runs and 300/300 requests, with real queueing (max waiting 7, max running 4), max KV usage about 2.84%, and 0 preemptions. But native chunking did not improve late-request TTFT

or produce the simulator-like reversal. Under the pre-specified interpretation criteria, this was a failed direct analogue rather than evidence that chunked prefill is ineffective.

6.3 Semantic diagnosis

The diagnosis found a semantic mismatch. Native vLLM V1 does not implement the same fixed-budget FCFS abstraction: running and waiting requests interact under native scheduler rules, and `max_num_batched_tokens` is itself a scheduling budget. Trace evidence showed overlap but not equivalence: FULL produced 6 mixed prefill+decode steps and 0 partial chunks, whereas CHUNKED produced 16 mixed steps and 21 partial chunks. The 0.5B model was not simply too fast to create prefill cost: a 3,200-token prefill took about 0.0767 s, versus 0.00591 s/token for representative decode, a 12.99 ratio. The conservative conclusion is simulator-vLLM semantic mismatch, motivating engine-level fidelity checks alongside simulator studies such as Vidur [1].

6.4 Native token-budget control

After diagnosis, we isolated a native vLLM control. Both treatments enabled chunked prefill and held model, manifest, warmup, sampler, and server controls fixed; only `max_num_batched_tokens` differed: 512 versus 4096. The run used two preselected regimes, 20 measured regime-runs, 150/150 successful requests, ABBA block order, and excluded warmups.

Figure 2 contrasts the simulator treatment, the bundled native analogue, and the controlled token-budget effect. Scheduler traces show separation: T512 executed 1,680 steps, 165 mixed steps, 194 partial-prefill items, and mean 436.1 prompt tokens per prefill step; T4096 executed 1,536 steps, 55 mixed steps, 9 partial-prefill items, and mean 1,535.9 prompt tokens per prefill step.

The latency effects were workload-dependent. In the low-late regime, T4096 improved late-request TTFT by 30.6 ms relative to T512, CI [-38.0, -22.8] ms, with 5/5 sign consistency, but worsened prompt-heavy E2E by 16.3 ms, CI [9.2, 23.5] ms. In the high-late regime, late-TTFT movement was smaller (-2.5 ms, CI [-7.7, 1.8] ms, 3/5 sign consistency), while prompt-heavy E2E again worsened by 23.3 ms, CI [15.6, 31.8] ms. Under the pre-specified interpretation criteria, this was a repeatable native token-budget effect in the tested configuration: vLLM exposed a workload-dependent scheduler-token-budget tradeoff, but not a Family-B reproduction. Serving simulators can reveal useful hypotheses, yet transfer to engines such as vLLM requires semantic checks against actual scheduler and budget controls [12, 2, 31, 19].

7 Implications, Limitations, Related Work, and Conclusion

Implications and limitations. The main implication is that complementarity is not exploitability. In the algorithm-selection sense, a VBS can expose portfolio

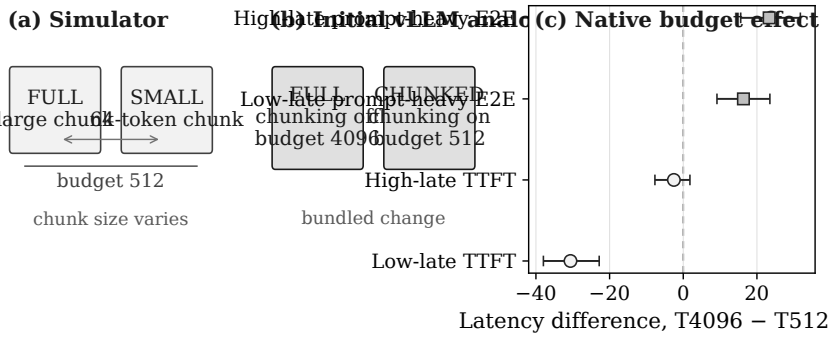

Fig. 2. Simulator-to-native vLLM semantic validation. (a) Fixed 512-token simulator budget with varying chunk size. (b) Initial native analogue changing chunking and `max_num_batched_tokens`. (c) Controlled native experiment; T4096 minus T512 latency differences with 95% bootstrap CIs. Negative late TTFT favors T4096; positive prompt-heavy E2E favors T512.

opportunity without implying that an online selector will capture much of the opportunity above the SBS. In LLM serving, scheduling choices change waiting times, active sequences, KV pressure, and future urgency, so regime prediction is not action-value prediction. High classification accuracy can coexist with small realized gain when policy advantage is sparse, labels are coarse, or errors fall on high-value states.

Portfolio construction is therefore only a first step. Future adaptive serving work may need action-value-aware state, counterfactual evaluation, stronger semantic composition, or closed-loop objectives that directly target deployable utility. Our negative results are bounded to the tested formulations.

The real-vLLM study adds a systems implication: mechanism validation requires checking scheduler semantics, admission rules, and control variables in the target engine rather than assuming similarly named controls transfer directly.

Several limitations bound the claims. The workloads remain synthetic despite the joint multi-pressure suite. One public replay saturated, but this does not make public traces generally uninformative. The six policies are mechanism-diverse repository baselines, not exhaustive reproductions of modern serving systems. The real-system evidence uses one vLLM version, one small model, one GPU, and one mechanism family. Finally, ANWG reflects a chosen weighted SLO-goodput objective; other objectives may change portfolio relationships.

Related work. LLM-serving work has developed strong mechanisms and architectures: Orca, vLLM/PagedAttention, Sarathi-Serve, DistServe, Llumnix, Mooncake, VTC, SOLA, FastServe, QLM, learning-to-rank schedulers, online KV scheduling, and WAIT/Nested WAIT [30, 12, 2, 31, 24, 19, 23, 8, 27, 18, 6, 9, 3]. Our study is complementary: it asks how much opportunity in a heterogeneous

scheduler portfolio is captured by tested adaptive mechanisms, not whether our baselines outperform these systems.

The methodology builds on algorithm selection and portfolios, including Rice, Gomes and Selman, SATzilla, AutoFolio, and Hydra [20, 7, 29, 13, 28]. Our exploitability analysis is an LLM-serving specialization of the SBS-versus-VBS perspective, not a new algorithm-selection concept. Hyper-heuristics and GP provide another precedent through classical scheduling rule selection and GP-generated rules [11, 15, 26, 4]; FunSearch and Autopoiesis show broader interest in automated program and serving-policy evolution [21, 10]. Our typed screen differs by using exact parent reproduction, a portfolio-envelope objective, and pre-specified nonredundancy criteria in an LLM-serving scheduler domain.

Conclusion. LLM-serving scheduler portfolios can contain real workload-dependent complementarity: the VBS retains measurable headroom over the SBS even under jointly varying multi-mechanism workloads. The central finding is that this opportunity is hard to exploit. The tested contextual selectors, router, support expansion, guarded composition, and structural synthesis did not convert VBS headroom into robust deployable gain under pre-specified criteria, and simulator mechanisms require engine-level semantic validation before transfer claims are made. Future adaptive LLM-serving work should optimize closed-loop utility directly rather than inferring deployable gain from VBS complementarity alone.

Acknowledgments. The author thanks Professor Ioannis Koutis for guidance and support, Anders Borum for lifetime access to Secure ShellFish supporting the research workflow, and the author's mother for continued support.

Data and Code Availability Code, workload generators, configurations, and derived artifacts are available at `https://github.com/SoroushVahidi/llm-serving-heuristic-evolution`. Simulator results use project-generated synthetic workloads and, for public-trace replay, derived windows from BurstGPT and Azure 2023 traces [25, 14, 17].

Generative AI Use The author used ChatGPT, Claude, Codex, and Perplexity AI for organization, language refinement, literature-search planning, software-development tasks, and workflow support. The author verified the designs, results, interpretations, citations, and final content, and takes responsibility for the work.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B.S., Ramjee, R., Tumanov, A.: Vidur: A large-scale simulation framework for LLM inference. In: Proceedings of Machine Learning and Systems. vol. 6 (2024)
2. Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B.S., Tumanov, A., Ramjee, R.: Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 117–134. USENIX Association (2024)

3. Ao, R., Luo, G., Simchi-Levi, D., Wang, X.: Optimizing LLM inference: Fluid-guided online scheduling with memory constraints. arXiv preprint arXiv:2504.11320 (2025)
4. Branke, J., Hildebrandt, T., Scholz-Reiter, B.: Hyper-heuristic evolution of dispatching rules: A comparison of rule representations. Evolutionary Computation **23**(2), 249–277 (2015)
5. Demers, A., Keshav, S., Shenker, S.: Analysis and simulation of a fair queueing algorithm. In: Proceedings of the ACM Symposium on Communications Architectures and Protocols (SIGCOMM). pp. 1–12. ACM (1989)
6. Fu, Y., Zhu, S., Su, R., Qiao, A., Stoica, I., Zhang, H.: Efficient LLM scheduling by learning to rank. In: Advances in Neural Information Processing Systems 37 (2024)
7. Gomes, C.P., Selman, B.: Algorithm portfolios. Artificial Intelligence **126**(1–2), 43–62 (2001)
8. Hong, K., Li, X., Chen, L., Mao, Q., Dai, G., Ning, X., Yan, S., Liang, Y., Wang, Y.: SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling. In: Proceedings of Machine Learning and Systems. vol. 7 (2025)
9. Jaillet, P., Jiang, J., Podimata, C., Zhou, Z.: Online scheduling for LLM inference with KV cache constraints. arXiv preprint arXiv:2502.07115 (2025)
10. Jiang, Y., Yan, R., Peng, Y., Li, W., Wang, T., Fu, F., Yuan, B.: Autopoiesis: An evolutionary system for automated design of LLM inference algorithms. arXiv preprint arXiv:2604.07144 (2026)
11. Koza, J.R.: Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press, Cambridge, MA (1992)
12. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with PagedAttention. In: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023). pp. 611–626. ACM (2023)
13. Lindauer, M., Hoos, H.H., Hutter, F., Schaub, T.: AutoFolio: An automatically configured algorithm selector. Journal of Artificial Intelligence Research **53**, 745–778 (2015)
14. Microsoft Azure: Azure LLM inference trace 2023. `https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md` (2023), public trace dataset accompanying Splitwise
15. Montana, D.J.: Strongly typed genetic programming. Evolutionary Computation **3**(2), 199–230 (1995)
16. Parekh, A.K., Gallager, R.G.: A generalized processor sharing approach to flow control in integrated services networks: The single-node case. IEEE/ACM Transactions on Networking **1**(3), 344–357 (1993)
17. Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, I., Maleki, S., Bianchini, R.: Splitwise: Efficient generative LLM inference using phase splitting. In: Proceedings of the 51st Annual International Symposium on Computer Architecture. IEEE Computer Society (2024). https://doi.org/10.1109/ISCA59077.2024.00019
18. Patke, A., Reddy, D., Jha, S., Qiu, H., Pinto, C., Narayanaswami, C., Kalbarczyk, Z.T., Iyer, R.K.: Queue management for large language model serving. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC 2024). ACM (2024)
19. Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., Xu, X.: Mooncake: Trading more storage for less computation – a KVCache-centric

architecture for serving LLM chatbot. In: 23rd USENIX Conference on File and Storage Technologies (FAST 25). pp. 155–170. USENIX Association (2025)
20. Rice, J.R.: The algorithm selection problem. Advances in Computers **15**, 65–118 (1976)
21. Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J.R., Ellenberg, J.S., Wang, P., Fawzi, O., Kohli, P., Fawzi, A.: Mathematical discoveries from program search with large language models. Nature **625**(7995), 468–475 (2024)
22. Schrage, L.: A proof of the optimality of the shortest remaining processing time discipline. Operations Research **16**(3), 687–690 (1968)
23. Sheng, Y., Cao, S., Li, D., Zhu, B., Li, Z., Zhuo, D., Gonzalez, J.E., Stoica, I.: Fairness in serving large language models. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 965–988. USENIX Association (2024)
24. Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., Lin, W.: Llumnix: Dynamic scheduling for large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 173–191. USENIX Association (2024)
25. Wang, Y., Chen, Y., Li, Z., Kang, X., Fang, Y., Zhou, Y., Zheng, Y., Tang, Z., He, X., Guo, R., Wang, X., Wang, Q., Zhou, A.C., Chu, X.: BurstGPT: A real-world workload dataset to optimize LLM serving systems. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 5831–5841. ACM (2025). https://doi.org/10.1145/3711896.3737413
26. Whigham, P.A.: Grammatically-based genetic programming. In: Proceedings of the Workshop on Genetic Programming: From Theory to Real-World Applications. pp. 33–41 (1995)
27. Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., Jin, X.: FastServe: Iteration-Level preemptive scheduling for large language model inference. In: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). pp. 57–74. USENIX Association (2026)
28. Xu, L., Hoos, H.H., Leyton-Brown, K.: Hydra: Automatically configuring algorithms for portfolio-based selection. In: Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence. pp. 210–216 (2010)
29. Xu, L., Hutter, F., Hoos, H.H., Leyton-Brown, K.: SATzilla: Portfolio-based algorithm selection for SAT. Journal of Artificial Intelligence Research **32**, 565–606 (2008)
30. Yu, G.I., Jeong, J.S., Kim, G.W., Kim, S., Chun, B.G.: Orca: A distributed serving system for transformer-based generative models. In: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). pp. 521–538. USENIX Association (2022)
31. Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H.: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 193–210. USENIX Association (2024)